

Food Fiesta

Developer Onboarding Guide

GENERATED ON 20 JULY 2026

1. Purpose

Food Fiesta is a full-stack dining management application that enables customers to browse a food menu, place orders, and track their order history, while administrators manage the product catalog, user accounts, and view all orders.

Business Domain

Food Fiesta operates in the restaurant/food delivery domain, providing a digital platform for menu management and order processing.

User Roles

- **Customer (User):** Can register, log in, browse products, search for specific dishes, place orders, and view their order history
- **Administrator (Admin):** Can manage the entire system including users, admins, products, and orders through a comprehensive dashboard
- **OAuth2 Users:** Customers can also register/login via Google OAuth2

Core Features

- **Product Catalog:** Browse and search Indian cuisine items (Biryani, Paneer dishes, Chicken dishes, Chinese fusion, North Indian specialties, desserts)
- **Order Management:** Place orders with automatic total calculation, view order history
- **User Management:** Self-registration, manual admin creation, profile updates
- **Admin Dashboard:** Centralized CRUD operations for all entities (users, admins, products, orders)
- **Authentication:** Dual-track login (admin vs customer) with optional Google OAuth2
- **API Documentation:** Interactive Swagger UI for all endpoints

2. How to Start the Application

Prerequisites

- **Java Development Kit (JDK) 21** - Required runtime version
- **Maven** - Optional (Maven Wrapper included in project)
- **Docker** - Optional (only for PostgreSQL deployment)

Running Locally with H2 (Default)

The application uses an in-memory H2 database by default, requiring no external database setup.

On Linux/macOS:

```
./mvnw spring-boot:run
```

On Windows PowerShell:

```
.\mvnw.cmd spring-boot:run
```

If `JAVA_HOME` is not set on Windows:

```
$env:JAVA_HOME="C:\Program Files\Java\jdk-21.0.11"
.\mvnw.cmd spring-boot:run
```

Local URLs

Service	URL	Purpose
Application Frontend	<code>http://localhost:8080/</code>	Main landing page and navigation
Swagger UI	<code>http://localhost:8080/swagger-ui/index.html</code>	Interactive API documentation
H2 Console	<code>http://localhost:8080/h2-console</code>	Database management interface

Default Credentials

Admin Account (seeded on startup):

Email: `admin@foodfiesta.com`

Password: `admin123`

H2 Database Console:

JDBC URL: `jdbc:h2:mem:foodfiesta`

Username: `sa`

Password: *(leave blank)*

Environment Variables

No environment variables are required for local H2 development. The following are only needed for PostgreSQL deployment:

- `SPRING_DATASOURCE_URL` - PostgreSQL connection string
- `SPRING_DATASOURCE_USERNAME` - Database username
- `SPRING_DATASOURCE_PASSWORD` - Database password

Optional: Google OAuth2 Setup

To enable Google login, update `src/main/resources/application.properties`:

1. Go to [Google Cloud Console](#)
2. Create an OAuth 2.0 Client ID
3. Add authorized redirect URI: `http://localhost:8080/login/oauth2/code/google`
4. Replace placeholders in `application.properties`:

```
spring.security.oauth2.client.registration.google.client-id=YOUR_CLIENT_ID
spring.security.oauth2.client.registration.google.client-secret=YOUR_CLIENT_SECRET
```

3. Application Architecture

Architectural Layers

Food Fiesta follows a classic Spring Boot layered architecture with some non-standard conventions:

Controllers (Entry Points)

Handle HTTP requests and route them to appropriate services. All controllers are annotated with `@Controller` and use Thymeleaf for server-side rendering.

- `HomeController` - Public pages (home, products, about, location, login, register)
- `AdminController` - Admin authentication, dashboard, and all CRUD operations
- `UserController` - User account management (add, update, delete, register)
- `ProductController` - Product CRUD operations
- `OrderController` - Currently empty (order logic is in AdminController)
- `OAuth2Controller` - Google OAuth2 callback handler
- `TestController` - Development/testing endpoints

Services (Business Logic)

 **Non-Standard Annotation:** All service classes use `@Component` instead of the conventional `@Service`. This is intentional and works correctly.

- `UserServices` - User CRUD, authentication validation (loads all users for comparison  not scalable)
- `AdminServices` - Admin CRUD, authentication validation,  **bug in update() method** (doesn't set ID before save)

- `ProductServices` - Product CRUD, search by name
- `OrderServices` - Order CRUD, fetch orders by user

Repositories (Data Access)

All repositories extend `CrudRepository` (not `JpaRepository`), meaning `findAll()` returns `Iterable` and must be cast to `List`.

- `UserRepository` - Custom query: `findUserByUemail(String email)`
- `AdminRepository` - Custom query: `findByAdminEmail(String email)`
- `ProductRepository` - Custom query: `findByPnameContainingIgnoreCase(String name)`
- `OrderRepository` - Custom query: `findOrdersByUser(User user)`

Entities (Data Model)

Entity	Table Name	ID Strategy	Notes
User	customer_info	IDENTITY	Field <code>u_id</code> uses underscore; getter is <code>getU_id()</code>
Admin	admin	IDENTITY	Passwords stored in plain text
Product	product_table	IDENTITY	Standard entity
Orders	order_info	AUTO	⚠ Field typo: <code>totalAmmout</code> (double 'm') - preserve this!

Relationships

- `User` ↔ `Orders` : One-to-Many (one user can have many orders)
- `Orders.user` : ManyToOne relationship with foreign key `user_u_id`

Configuration

- `SecurityConfig` - Spring Security configuration (CSRF disabled, route authorization, OAuth2 setup)
- `OpenApiConfig` - Swagger/OpenAPI documentation setup
- `DataLoader` - Seeds 5 sample products and 1 default admin on startup (guarded by `count() == 0`)

Utilities

- `Logic` (in `count` package) - Static utility for order total calculation: `countTotal(price, quantity)`
- `AdminLogin` and `UserLogin` (in `loginCredentials` package) - Plain DTOs for Thymeleaf form binding (not entities)

Authentication Mechanism

⚠ **Critical Security Note:** Spring Security only guards routes. Actual authentication is manual and session-based.

How Authentication Works

1. Admin Login:

- User submits credentials to `/adminLogin` (POST)

- `AdminController.getAllData()` validates via `AdminServices.validateAdminCredentials()`
- If valid, stores admin email as `String` in session: `session.setAttribute("loggedInAdmin", email)`
- Redirects to `/admin/services`

2. User Login:

- User submits credentials to `/userLogin` (POST)
- `AdminController.userLogin()` validates via `UserServices.validateLoginCredentials()`
- If valid, stores full `User` object in session: `session.setAttribute("loggedInUser", user)`
- Redirects to `/dashboard`

3. OAuth2 Login:

- Google redirects to `/oauth2/success`
- `OAuth2Controller` extracts email from OAuth2 principal
- Creates new user if not exists (password = `"OAUTH_USER"`, phone = `0L`)
- Stores `User` object in session
- Redirects to `/dashboard`

Password Storage

⚠ **Security Gap:** All passwords (admin and user) are stored in plain text. A `BCryptPasswordEncoder` bean exists in `SecurityConfig` but is not used anywhere in the codebase.

Session Keys

- `"loggedInAdmin"` → `String` (admin email)
- `"loggedInUser"` → `User` (full JPA entity)

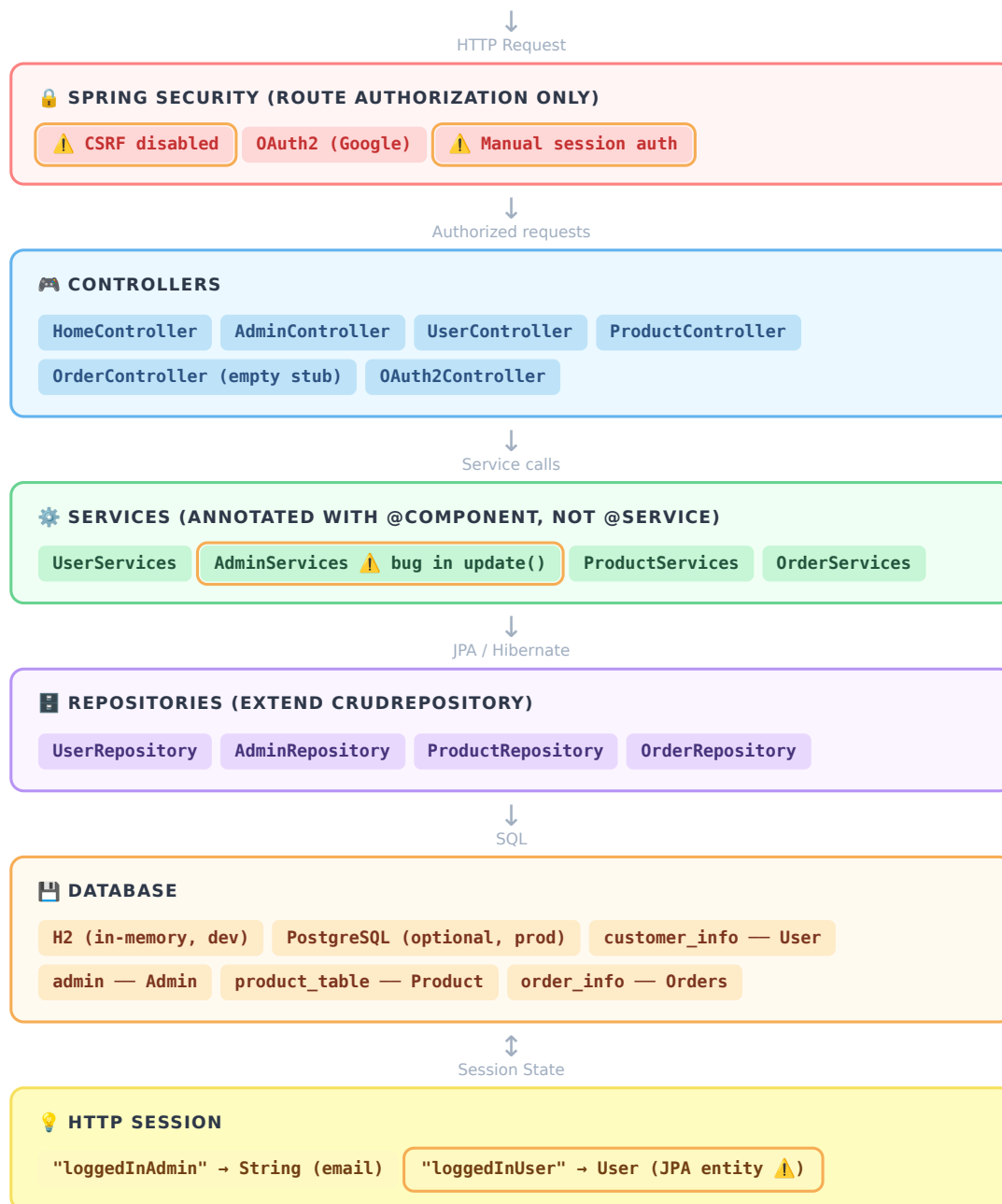
Request Lifecycle

1. **Browser** sends HTTP request (Thymeleaf form submission or direct navigation)
2. **Spring Security** checks if route is public or requires authentication
3. **Controller** receives request, validates session if needed
4. **Service** executes business logic (validation, calculation)
5. **Repository** performs database operations via Spring Data JPA
6. **H2/PostgreSQL** executes SQL and returns data
7. **Service** processes results
8. **Controller** adds data to Model and returns Thymeleaf template name
9. **Thymeleaf** renders HTML with data
10. **Browser** displays the page

Architecture Diagram

🌐 **CLIENT / BROWSER**

Thymeleaf SSR HTML Forms CSS + JavaScript



4. CI/CD & Deployment

Continuous Integration - AWS CodeBuild

The project uses AWS CodeBuild for CI, configured in `buildspec.yml`.

Build Phases

- **Install:** Displays Maven and Java versions
- **Build:** Executes `mvn clean package -DskipTests`
- **Post-Build:** Copies JAR to `output/` directory

Artifacts Produced

- `output/*.jar` - Compiled Spring Boot application

- `appspec.yml` - AWS CodeDeploy configuration (if present)
- `scripts/**/*` - Deployment scripts (if present)

Note: Tests are skipped during CI build (`-DskipTests`). This is because tests require Spring context with valid OAuth2 credentials, which are placeholders in `application.properties` .

Docker

Dockerfile (Multi-Stage Build)

The Dockerfile uses a two-stage build for optimal image size:

Stage	Base Image	Purpose
Build	<code>eclipse-temurin:21-jdk-alpine</code>	Compile application with Maven
Run	<code>eclipse-temurin:21-jre-alpine</code>	Minimal runtime environment

Security Measures

- Non-root user: Application runs as `spring:spring` user
- Minimal runtime: JRE-only image (no JDK in production)
- Alpine Linux: Smaller attack surface

Environment Variables (Dockerfile)

Variable	Default Value	Purpose
<code>SPRING_DATASOURCE_URL</code>	<code>jdbc:postgresql://db:5432/foodfiesta</code>	PostgreSQL connection string
<code>SPRING_DATASOURCE_USERNAME</code>	<code>postgres</code>	Database username
<code>SPRING_DATASOURCE_PASSWORD</code>	<code>password</code>	Database password

Docker Compose

`docker-compose.yml` orchestrates two services:

Service	Image	Ports	Notes
<code>app</code>	Built from Dockerfile	8080:8080	Waits for <code>db</code> health check
<code>db</code>	<code>postgres:latest</code>	5432:5432	Health check: <code>pg_isready -U postgres</code>

Entrypoint Script

`entrypoint.sh` is a simple startup script that launches the JAR:

```
#!/bin/sh
echo "🚀 Starting Food Fiesta Application..."
```

```
exec java -jar app.jar
```

Environment Variables for Production

Variable	Required	Purpose	Default (Dev)
<code>SPRING_DATASOURCE_URL</code>	Yes (if using PostgreSQL)	Database connection string	<code>jdbc:h2:mem:foodfiesta</code>
<code>SPRING_DATASOURCE_USERNAME</code>	Yes (if using PostgreSQL)	Database username	<code>sa</code>
<code>SPRING_DATASOURCE_PASSWORD</code>	Yes (if using PostgreSQL)	Database password	<i>(empty)</i>
<code>spring.security.oauth2.client.registration.google.client-id</code>	No (optional OAuth2)	Google OAuth2 client ID	<code>YOUR_CLIENT_ID</code>
<code>spring.security.oauth2.client.registration.google.client-secret</code>	No (optional OAuth2)	Google OAuth2 client secret	<code>YOUR_CLIENT_SECRET</code>

Health Check Endpoint

No dedicated health check endpoint is currently implemented. Consider adding Spring Boot Actuator for production monitoring:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

This would enable `/actuator/health` endpoint for container health checks.

5. Project Health & Technical Debt Tracking

To help you monitor the project's health over time, a technical debt audit will be automatically run at the end of this onboarding process. This audit provides a comprehensive analysis of code quality, security vulnerabilities, architectural issues, and dependency risks.

Audit Deliverables

The audit generates three files in the `docs/audit/` directory:

- `dette_technique.html` - Interactive HTML report with KPI dashboard and prioritized backlog
- `dette_technique.pdf` - PDF version with debt visualization chart
- `dette_technique_baseline.json` - Temporal tracking baseline for future comparisons

Re-Running the Audit

You can regenerate the audit at any time to track improvements or regressions:

Type **"lance un audit de dette technique"** in a new Bob conversation on this project to regenerate the full audit report with temporal delta (improvement or regression vs. the previous baseline)

Temporal Tracking

The baseline JSON file (`docs/audit/dette_technique_baseline.json`) stores a reference snapshot of the project's health metrics. When you re-run the audit, it automatically compares against this baseline and shows:

- Which metrics have improved (e.g., reduced security vulnerabilities)
- Which metrics have regressed (e.g., increased code complexity)
- Net change in technical debt (days and euros)

Best Practice: Run the audit before and after major refactoring efforts to quantify the impact of your improvements.

6. Known Gotchas & Contribution Guide

Known Gotchas

 **Read this section carefully before making any changes!**

1. Intentional Typo in Orders Entity

Field: `Orders.totalAmmout` (note the double 'm')

Why it exists: This field maps to a database column. Changing the spelling would break JPA column mapping.

Action: Never "fix" this typo. Use `getTotalAmmout()` and `setTotalAmmout()` as-is.

2. Services Use @Component Instead of @Service

Classes: `UserServices` , `AdminServices` , `ProductServices` , `OrderServices`

Why it exists: This is an intentional architectural choice. Both annotations work for Spring bean registration.

Action: Do not change `@Component` to `@Service` . Maintain consistency with existing code.

3. Unsafe Optional.get() Calls

Pattern: `Optional<User> optional = repository.findById(id); User user = optional.get();`

Risk: Throws `NoSuchElementException` if ID doesn't exist.

Action: When adding new code, use `optional.orElseThrow()` or `optional.orElse(null)` with proper null checks.

4. Update Operations Use @GetMapping

Routes: `/updatingAdmin/{id}`, `/updatingProduct/{productId}`, `/updatingUser/{id}`

Why it exists: This is the established pattern in the codebase (not RESTful, but intentional).

Action: Follow the existing pattern for consistency. Do not change to `@PutMapping` without team discussion.

5. AdminServices.update() Bug

Issue: The method iterates all admins to find a match but calls `save(admin)` without setting the ID on the passed-in admin object.

```
public void update/Admin admin, int id {
    for (Admin ad : getAll()) {
        if(ad.getAdminId()==id) {
            this.adminRepository.save(admin); // ⚠ admin.adminId is not set!
        }
    }
}
```

Risk: The saved record may get a new ID instead of updating the existing one.

Action: If you need to fix this, add `admin.setAdminId(id);` before the save call.

6. Passwords Stored in Plain Text

Issue: Both admin and user passwords are stored without hashing.

Why it exists: A `BCryptPasswordEncoder` bean exists but is not wired to any authentication logic.

Action: Do not change password storage without migrating existing data or handling both formats during transition.

7. Session Stores Full JPA Entity

Pattern: `session.setAttribute("loggedInUser", user)` stores a full `User` object.

Risk: If the session object becomes detached from the Hibernate session, lazy-loaded fields (like `orders`) will throw `LazyInitializationException`.

Action: Be cautious when accessing `user.getOrders()` from session. Consider eager fetching or re-attaching the entity.

8. UserServices.validateLoginCredentials() Loads All Users

Issue: The method loads all users into memory and iterates to find a match.

Risk: Not scalable for large user bases.

Action: Before adding significant user load, replace with a custom query method like `findUserByUemailAndUpassword(String email, String password)`.

9. CrudRepository Returns Iterable, Not List

Pattern: All repositories extend `CrudRepository`, so `findAll()` returns `Iterable<T>`.

Action: Always cast to `List`: `List<User> users = (List<User>) repository.findAll();`

10. Orders Uses GenerationType.AUTO

Issue: `Orders` uses `GenerationType.AUTO` while all other entities use `IDENTITY`.

Risk: Can cause sequence table conflicts when switching to PostgreSQL.

Action: Normalize to `IDENTITY` before production migration if using PostgreSQL.

11. Default Admin Credentials

Credentials: `admin@foodfiesta.com` / `admin123`

Action: Do not change these without updating the team. They are seeded on every fresh startup.

12. H2 Database is In-Memory

Behavior: All data is lost when the application stops.

Action: Use PostgreSQL (via Docker Compose) if you need persistent data during development.

Contribution Guide

Running Tests Locally

```
./mvnw test
```

On Windows:

```
.\mvnw.cmd test
```

⚠ Test Requirement: All existing tests must pass before pushing. Currently, only one test exists (`FoodFrenzyApplicationTests`) which checks that the Spring context loads.

Branching Convention

Based on the project structure, the following convention is recommended:

- Main branch: `main`
- Feature branches: `feature/description`
- Bugfix branches: `bugfix/description`
- Merge via Pull Request (PR) with code review

Running the App Before Pushing

```
./mvnw spring-boot:run
```

Verify that:

- Application starts without errors
- You can access `http://localhost:8080/`
- Login functionality works (admin and user)
- Your changes work as expected

CI Checks (AWS CodeBuild)

When you push, AWS CodeBuild will automatically:

- Verify Maven and Java versions
- Run `mvn clean package -DskipTests`
- Produce a JAR artifact

Adding New Service Methods

Reminder: If you add a new service method, add a matching test in the corresponding `*Test.java` file (even if it's just a basic smoke test).

Code Style

- Follow existing naming conventions (e.g., `u_id`, `totalAmmout`)
- Use `@Component` for services (not `@Service`)
- Add Swagger annotations to new controller methods: `@Operation(summary="...", description="...")`
- Cast `findAll()` results to `List` explicitly
- Use `Logic.countTotal()` for order calculations (never inline price × quantity)